

CAR SERVICE BOOKING SYSTEM

Microservices Architecture — Capstone Project

Software Requirements Specification & Trainee Implementation Guide

Prepared for: Java Full Stack DSE Trainees

1. Project Overview

The **Car Service Booking System** is a web-based **microservices application** that enables customers to register, manage their vehicles, browse available car services, and book service appointments online. The system provides a platform for managing customer information, service catalogs, and service bookings through a structured and user-friendly interface.

The application supports two user roles: **Administrator** and **Customer**. Customers can manage their profiles, vehicles, browse available services, create service bookings, and track their booking history. Administrators manage service categories, car services, customer bookings, and monitor overall system activities through dashboards and reports.

The project is developed using **Java 21, Spring Boot, Spring Security, Spring Data JPA, Spring Cloud Gateway, Spring Cloud Consul, Angular 15, TypeScript, MySQL, and Maven**. It is designed to evaluate trainees' understanding of Java Full Stack development, REST APIs, Microservices Architecture, database design, and modern software engineering practices through the implementation of a real-world application.

1.1 Starter Code Notice

A starter project is provided to establish the application foundation and reduce setup time. The starter kit includes the project structure, common configurations, authentication module, selected CRUD modules, database scripts, and the frontend application skeleton.

The following user stories are **fully implemented** in the starter project:

- **US01 – Authentication & Authorization**
- **US02 – Customer Profile Management**
- **US03 – Vehicle Management**
- **US04 – Service Category Management**

Trainees are expected to implement the remaining user stories (**US05–US10**) by extending the provided codebase while following the implementation guidelines, coding standards, and acceptance criteria defined in this document.

2. System Architecture

The solution follows a Microservices Architecture consisting of Authentication Service, User Service, Service Catalog Service, and Booking Service, with communication managed through Spring Cloud Gateway and Spring Cloud Consul. All frontend requests must pass through the API Gateway, which is responsible for routing requests to the appropriate backend services.

2.1 Infrastructure Mapping

Service	Port	Database Schema
API Gateway	8080	None
Consul (Discovery)	8500	None
Auth Service	8081	auth_db

Service	Port	Database Schema
User Service	8082	user_db
Catalog Service	8083	catalog_db
Booking Service	8084	booking_db
Angular Frontend	4200	LocalStorage Session

2.2 Important Development Rule

API Gateway Communication: The Angular frontend must communicate only with `http://localhost:8080/api/...`. Do not hardcode direct microservice ports such as 8081, 8082, etc.

3. User Stories

The following ten user stories define the complete functional scope of the application. US01–US04 are provided as part of the starter codebase; US05–US10 are the trainee implementation scope (see Section 5).

US 01 – Authentication & Authorization

Priority: Must Have

Primary Microservice: Authentication Service

User Story Description

As a **Customer or Administrator**, I want to securely register and log in so that I can access the application based on my assigned role.

Acceptance Criteria

- Customers shall be able to register using Full Name, Email Address, and Password.
- Email addresses shall be unique across the system.
- Users shall be able to log in using their Email Address and Password.
- Authentication shall be performed by validating the entered credentials against the stored records.
- User role shall be identified during successful login.
- Role-based access shall be enforced throughout the application.
- Users shall be able to log out by clearing the active session.
- Duplicate email registrations shall not be allowed.
- Mandatory fields shall be validated before registration and login.
- Appropriate success, validation, and error messages shall be displayed.

US 02 – Customer Profile Management

Priority: Must Have

Primary Microservice: User Service

User Story Description

As a **Customer**, I want to manage my profile information so that my personal details remain accurate for future service bookings.

Acceptance Criteria

- Customers shall be able to view their profile information.
- Customers shall be able to update their profile details.
- Profile information shall include User ID, Full Name, Email Address, Contact Number, and Address.
- User ID shall be displayed but shall not be editable.
- Email addresses shall remain unique.
- Customers shall only access profiles associated with their own User ID.
- Administrators shall have read-only access to customer profiles.
- Updated profile information shall be reflected immediately after saving.
- Mandatory fields shall be validated before saving.
- Appropriate success, validation, and error messages shall be displayed.

US 03 – Vehicle Management

Priority: Must Have

Primary Microservice: User Service

User Story Description

As a **Customer**, I want to manage my vehicles so that I can select the appropriate vehicle while booking a car service.

Acceptance Criteria

- Customers shall be able to add, view, update, and delete vehicles.
- Each vehicle shall contain Registration Number, Manufacturer, Model, Fuel Type, Manufacturing Year, and Color.
- Fuel Type shall support the following values:
 - PETROL
 - DIESEL
 - CNG
 - ELECTRIC
 - HYBRID

- Vehicle registration numbers shall be unique.
- Vehicle ownership shall automatically be associated with the logged-in customer.
- Customers shall only manage their own vehicles.
- Administrators shall have read-only access to vehicle information.
- Vehicles shall support searching, sorting, and pagination.
- Vehicles linked to active bookings shall not be deleted.
- Appropriate success, validation, and error messages shall be displayed.

US 04 – Service Category Management

Priority: Must Have

Primary Microservice: Service Catalog Service

User Story Description

As an **Administrator**, I want to manage service categories so that car services can be organized for easy customer navigation.

Acceptance Criteria

- Administrator shall be able to add, view, update, and delete service categories.
- Each category shall contain Category Name and Description.
- Duplicate category names shall not be allowed.
- Only Administrator shall manage service categories.
- Customers shall be able to view all available categories.
- Categories shall support searching, sorting, and pagination.
- Categories containing active services shall not be deleted.
- Mandatory fields shall be validated before saving.
- Appropriate success, validation, and error messages shall be displayed.
- Changes to service categories shall be immediately reflected in the application.

US 05 – Car Service Management

Priority: Must Have

Primary Microservice: Service Catalog Service

User Story Description

As an **Administrator**, I want to manage car services so that customers can browse and book available services.

Acceptance Criteria

- Administrator shall be able to add, view, update, and delete car services.
- Every service shall belong to a valid service category.
- Each service shall contain Service Name, Description, Estimated Duration, Price, and Availability Status.
- Availability Status shall support the following values:
 - ACTIVE
 - INACTIVE
- Only Administrator shall manage car services.
- Administrator shall be able to activate or deactivate services.
- Customers shall have read-only access to active services.
- Services shall support searching, sorting, and pagination.
- Services associated with active bookings shall not be deleted.
- Appropriate success, validation, and error messages shall be displayed.

US 06 – Browse & Search Services

Priority: Must Have

Primary Microservice: Service Catalog Service

User Story Description

As a **Customer**, I want to browse and search available car services so that I can choose the most suitable service for my vehicle.

Acceptance Criteria

- Customers shall be able to view all available service categories.
- Customers shall be able to browse services under each category.
- Customers shall be able to search services using the Service Name.
- Customers shall be able to filter services by Category.
- Customers shall be able to sort services by Service Name or Price.
- Service listings shall support pagination.
- Service details shall display Service Name, Description, Estimated Duration, Price, and Availability Status.
- Only services with **ACTIVE** availability status shall be visible to customers.
- Appropriate messages shall be displayed when no services are available.
- Changes made by the Administrator shall be reflected immediately in customer service listings.

US 07 – Car Service Booking

Priority: Must Have

Primary Microservice: Booking Service

User Story Description

As a **Customer**, I want to book a service appointment for my registered vehicle so that I can schedule maintenance or repair services.

Acceptance Criteria

- Customers shall be able to create a booking by selecting Vehicle, Service, Appointment Date, Time Slot, and Problem Description (Optional).
 - Customers shall only be able to select vehicles associated with their own User ID.
 - Customers shall only be able to book services with **ACTIVE** availability status.
 - Appointment dates shall not be in the past.
 - Duplicate bookings for the same vehicle, appointment date, and time slot shall not be allowed.
 - Newly created bookings shall have the default status **PENDING**.
 - Customers shall be able to view booking confirmation immediately after successful booking.
 - Customers shall be able to cancel only bookings having **PENDING** status.
 - Mandatory fields shall be validated before creating a booking.
 - Appropriate success, validation, and error messages shall be displayed.
-

US 08 – Booking Management

Priority: Must Have

Primary Microservice: Booking Service

User Story Description

As an **Administrator**, I want to manage customer bookings so that service appointments can be processed efficiently.

Acceptance Criteria

- Administrator shall be able to view all customer bookings.
 - Booking details shall display Customer, Vehicle, Service, Appointment Date, Time Slot, Booking Status, and Problem Description.
 - Administrator shall be able to update booking status.
 - Booking status shall follow the workflow:
 - PENDING → CONFIRMED → IN_SERVICE → READY_FOR_DELIVERY → COMPLETED
 - PENDING → CANCELLED
 - Invalid booking status transitions shall not be allowed.
 - Customers shall be able to view the latest booking status.
 - Bookings shall support searching, sorting, and pagination.
 - Only Administrator shall be authorized to update booking status.
 - Appropriate success, validation, and error messages shall be displayed.
 - Updated booking status shall be reflected immediately to both Administrator and Customer.
-

US 09 – Dashboard & Booking History

Priority: Must Have

Primary Microservices: User Service, Service Catalog Service, Booking Service

User Story Description

As a **Customer or Administrator**, I want to view dashboard information and booking history so that I can quickly access important information related to my activities.

Acceptance Criteria

- Customers shall be able to view their booking history.
- Customer dashboard shall display summary information including Total Vehicles, Total Bookings, Active Bookings, and Completed Bookings.
- Administrator dashboard shall display Total Customers, Total Categories, Total Services, and Total Bookings.
- Administrator shall be able to view recent booking activities.
- Booking history shall support searching, filtering, sorting, and pagination.
- Customers shall only access booking history associated with their own User ID.
- Dashboard information shall always display the latest available data.
- Appropriate messages shall be displayed when no records are available.
- Dashboard information shall be displayed based on the logged-in user's role.
- Newly created or updated bookings shall be reflected immediately in the dashboard and booking history.

US 10 – Booking Search & Reports

Priority: Must Have

Primary Microservice: Booking Service

User Story Description

As an **Administrator**, I want to search and monitor booking records so that I can efficiently manage customer service requests.

Acceptance Criteria

- Administrator shall be able to search bookings using Booking Reference Number or Customer Name.
- Administrator shall be able to filter bookings by Booking Status and Appointment Date.
- Customers shall be able to search their own booking history.
- Search results shall support sorting and pagination.
- Booking statistics shall display the total number of bookings grouped by booking status.
- Customers shall only access booking records associated with their own User ID.

- Appropriate messages shall be displayed when no matching records are found.
- Search and report information shall always display the latest available booking data.
- Appropriate validation and exception handling shall be implemented.
- Search and report results shall be displayed based on the logged-in user's role.

3.1 Final User Story List

User Story	Module	Microservice
US01	Authentication & Authorization	Authentication Service
US02	Customer Profile Management	User Service
US03	Vehicle Management	User Service
US04	Service Category Management	Service Catalog Service
US05	Car Service Management	Service Catalog Service
US06	Browse & Search Services	Service Catalog Service
US07	Car Service Booking	Booking Service
US08	Booking Management	Booking Service
US09	Dashboard & Booking History	Booking, User & Service Catalog Services
US10	Booking Search & Reports	Booking Service

Data Persistence Rules

The application shall use the respective microservice databases as the permanent storage for all business data.

The following rules shall be followed throughout the application:

- All successful Create, Update and Delete operations shall be immediately persisted in the corresponding microservice database.
- All View and Search operations shall always retrieve the latest persisted records.
- Changes performed by one module shall be immediately reflected wherever the same data is consumed.
- No business data shall be maintained only in frontend memory.
- Data shall remain available after browser closure, frontend restart, backend restart, application restart, or system reboot.
- Each microservice shall manage only its own database schema.
- The database shall always be considered the single source of truth for all business operations.

Identifier Generation Rules

The system shall automatically generate unique identifiers for all business entities. The generated identifiers shall remain unique throughout the application lifecycle.

Entity	Format	Example
Administrator	A<number>	A1
Customer	U<number>	U1, U2, U3
Vehicle	V<number>	V1, V2, V3
Service Category	C<number>	C1, C2, C3
Car Service	S<number>	S1, S2, S3
Booking	B<number>	B1, B2, B3

These identifiers shall be generated automatically by the respective microservice during record creation and shall not be editable by end users.

4. Frontend Components

Authentication Module

Handles user registration, login, authentication, and session management.

Features

- User Registration
- User Login
- Role-Based Route Guards
- Session Management
- Logout Functionality

Dashboard Module

Displays role-based dashboard information for Customers and Administrators.

Features

- Dashboard Summary Cards
- Recent Bookings
- Booking History
- Booking Statistics
- Quick Navigation
- Responsive Dashboard
- Role-Based Dashboard View

Customer Profile Module

Manages customer profile information.

Features

- View Profile
 - Update Profile
 - Form Validation
 - Responsive Forms
 - Profile Information Display
-

Vehicle Management Module

Manages customer vehicle information.

Features

- Vehicle CRUD
- Vehicle List
- Search
- Sorting
- Pagination
- Vehicle Details View
- Delete Confirmation.

Service Category Module

Manages service categories.

Features

- Category CRUD
 - Category List
 - Search
 - Sorting
 - Pagination
 - Delete Confirmation
-

Car Service Module

Manages car services.

Features

- Service CRUD

- Browse Services
 - Search
 - Filter
 - Category-wise Listing
 - Sorting
 - Pagination
 - Service Details View
 - Availability Status (ACTIVE / INACTIVE)
-

Booking Module

Manages customer service bookings.

Features

- Create Booking
- Booking Summary
- Appointment Date Selection
- Time Slot Selection
- Problem Description
- Booking History
- Booking Cancellation
- View All Bookings (Admin)
- Update Booking Status (Admin)
- Booking Status Workflow

Reports Module

Provides booking search and reporting functionality.

Features

- Search by Booking Reference Number
 - Search by Customer Name
 - Filter by Booking Status
 - Filter by Appointment Date
 - Booking Statistics
 - Sorting
 - Pagination
 - Empty State Handling
-

Shared Core Module

Provides reusable UI components and common application services.

Features

- Global Header
- Navigation Menu
- Dynamic Sidebar
- Application Footer
- Route Guards
- HTTP Interceptors
- Shared Components
- API Services

Backend Components

Authentication Service

Responsibilities

- Registration
 - Login
 - Authentication
 - Role Validation
-

User Service

Responsibilities

- Customer Profile
 - Vehicle Management
-

Service Catalog Service

Responsibilities

- Category Management
 - Service Management
 - Browse Services
-

Booking Service

Responsibilities

- Booking
- Booking Management
- Dashboard Data

- Reports

5. Trainee Implementation Tasks (US05–US10)

Your objective is to complete the remaining business requirements by implementing User Stories US05 through US10 on top of the provided project skeleton. Full acceptance criteria for each user story are defined in Section 3.

5.1 Catalog Management (US05 & US06)

Location: `catalog-service` and `frontend/features/service`, `frontend/features/category`

- Implement Service Management for administrators (Add, Update and Manage services).
- Implement browsing and searching of active services.
- Complete repository queries and bind the data to Angular tables.

5.2 Booking Workflow (US07 & US08)

Location: `booking-service` and `frontend/features/booking`

- Allow users to book services for registered vehicles.
- Validate requests before saving into `booking_db`.
- Enable administrators to manage booking status (Pending → Confirmed → Completed).

5.3 Dashboards & Reporting (US09 & US10)

Location: `frontend/features/dashboard` and `frontend/features/reports`

- Display upcoming and previous bookings for logged-in users.
- Provide administrator reports with filtering by date range and booking status.

5.4 Code Navigation

Search the project for comments in the following format to identify the exact locations where implementation is expected:

```
| // TODO: Trainee to implement USXX
```

6. Getting Started

6.1 Setup Steps

1. Start Consul for service discovery.
2. Ensure MySQL is running on port 3306.
3. Start each backend microservice using `mvn clean spring-boot:run` or your IDE.
4. Start the Angular application by running `npm install` followed by `npm start`.
5. Login using the credentials in Section 6.2.

6.2 Default Login Credentials

Role	Email	Password
Admin	admin@gmail.com	admin1234
User	user@gmail.com	user1234

6.3 Testing Recommendation

Before integrating any feature with Angular, verify the backend endpoint using Postman or Swagger (<http://localhost:<port>/swagger-ui.html>).

7. Environment Requirements

7.1 Technology Stack

Frontend

- Angular 15
- TypeScript
- HTML5
- CSS3
- Bootstrap 5

Backend

- Java 21
- Spring Boot
- Spring Data JPA
- Spring Security
- Spring Cloud Gateway
- Spring Cloud Consul
- Spring Cloud Circuit Breaker (Resilience4j)
- REST APIs
- Lombok
- ModelMapper

Database

- MySQL

8. Non-Functional Requirements

- **Performance:** The application should be fast and responsive, with quick load times and minimal lag or delay.
- **Scalability:** The application should be able to handle large amounts of traffic and scale as needed to accommodate growth.
- **Reliability:** The application should be reliable and available, with minimal downtime or outages.
- **Usability:** The application should be easy to use and intuitive, with a clear and consistent user interface.
- **Maintainability:** The application should be easy to maintain and update, with clear and well-organized code that is easy to understand and modify.
- **Compatibility:** The application should be compatible with a wide range of devices and browsers, with support for different screen sizes and resolutions.
- **Implement caching mechanisms** to improve performance.
- **Lazy Loading:** Implement lazy loading techniques for images and other content, ensuring faster initial page loads and smoother user experience.
- **User Interface (UI) and User Experience (UX) Design:** Design an intuitive user interface that is easy to navigate and provides a seamless user experience. Prioritize responsiveness and clarity across all devices (desktop, mobile, tablets).

9. Implementation Guidelines

9.1 General Coding Practices

- Utilize Lombok for model class creation and logging.
- Implement logic using Lambdas and Streams.
- Organize code into logical layers (Controller, Service, Repository, DTO, Entity, Configuration, etc.).
- Use meaningful package names reflecting the domain.
- Employ Spring Dependency Injection for effective component management.

9.2 Error Handling

- Handle cases where the user attempts to access resources without authorization.
- Implement custom exceptions with user-friendly error messages.
- Use a centralized exception handling mechanism to translate exceptions into appropriate HTTP responses.
- Log all service exceptions using LoggingAspect.
- Return meaningful success and error responses with appropriate HTTP status codes.

9.3 DTOs and Mapping

- Data Transfer Objects (DTOs) should be used for API requests and responses.
- Entities should not be exposed directly through REST APIs.
- Use ModelMapper to convert between Entities and DTOs.

9.4 Database Interaction

- Spring Data Repository should be used for all database operations.
- Configure appropriate database properties and table generation strategies.
- Each microservice should maintain its own database schema to ensure loose coupling.

9.5 API Design

- All APIs should be mapped with an appropriate base URI.
- Configure server port numbers for each microservice.
- Use appropriate HTTP methods and status codes for all REST endpoints.
- Use Swagger/OpenAPI for API documentation.

9.6 Microservice Communication

- Register all microservices with Spring Cloud Consul for service discovery.
- Route all client requests through Spring Cloud Gateway.
- Use REST APIs for inter-service communication.
- Implement Spring Cloud Circuit Breaker (Resilience4j) for all critical service calls with a timeout of 3 seconds and an error threshold of 50%.
- The circuit breaker should open after 10 consecutive failures and remain open for 60 seconds.
- Define appropriate fallback methods for all critical operations.
- Create two instances of each business microservice and implement client-side load balancing using Spring Cloud LoadBalancer.

9.7 Validation

- Bean Validation should be used for all user inputs.
- Custom Validators should be implemented for complex validations.
- All request parameters must be validated for null or empty values. Invalid values should return the message: "Please provide a valid <attribute name>".

9.8 Testing

- Write JUnit test cases for all service methods using Mockito.
- Achieve a minimum of 80% code coverage.
- Test all REST APIs using Postman or Swagger/OpenAPI.

10. Code Quality

To ensure adherence to coding standards, the project should be analyzed using SonarQube. The following minimum acceptable values must be met:

SonarQube Metric	Minimum Acceptable Value
Security	A
Reliability	A

SonarQube Metric	Minimum Acceptable Value
Issues	≤ 5
Coverage	$\geq 80\%$
Duplications	$\leq 3\%$
Security Hotspots	A

10.1 REST API Testing

Once the implementation is completed, all REST endpoints should be tested using Postman or Swagger/OpenAPI to verify that the application functions correctly.

11. Final Deliverables

- Application archive (.jar) with source code
- Database DDL Script
- Complete Source Code
- Sample screenshots of important screens

Good Luck! Build clean, maintainable, and well-tested code.